Module 6: Control Flow - Loops

6.1 Why Loops Are Needed

- Repeating code manually is impractical when the repetition count is large or unknown
- A loop is a structure that repeats a block of code while a condition remains true
- Key terms
  - Iteration: one pass through the loop body
  - Loop condition: the boolean expression checked before (or after) each iteration
  - Loop variable or counter: a variable used to control or count iterations

6.2 The for Loop

- Best used when the number of iterations is known in advance
- Syntax: for (initialization; condition; update) { loop body }
- Three parts
  - Initialization: executed once before the loop starts (example: int i = 0)
  - Condition: checked before each iteration; loop runs as long as it is true
  - Update: executed after each iteration (example: i++ or i--)
- Counting from 1 to 10

```
for (int i = 1; i <= 10; i++) {
cout << i << " ";
}
```

- Counting down from 10 to 1

```
for (int i = 10; i >= 1; i--) {
cout << i << " ";
}
```

- Counting in steps of 2

```
for (int i = 0; i <= 20; i += 2) {
cout << i << " ";
}
```

- Common mistake: putting a semicolon after for(...)
  - for (int i = 0; i < 5; i++); means the loop body is empty and only the semicolon repeats
- The loop variable declared inside for is local to the loop and cannot be used after it

6.3 The while Loop

- Best used when the number of iterations is not known in advance
- The condition is checked BEFORE each iteration (pre-test loop)

- Syntax

```
while (condition) {
// loop body
}
```

- The loop body only runs if the condition is true from the start
- If the condition is false from the beginning, the loop body never executes
- Example: count from 1 to 5

```
int i = 1;
while (i <= 5) {
cout << i << " ";
i++;
}
```

- Infinite loop: when the condition never becomes false
- Always make sure the update inside the loop eventually makes the condition false
- Example of accidental infinite loop: forgetting to increment i inside the while loop
- Input validation with while loop
- Keep asking the user for input until they enter a valid value

```
int age;
cout << "Enter age (1-120): ";
cin >> age;
while (age < 1 || age > 120) {
cout << "Invalid. Try again: ";
cin >> age;
}
```

6.4 The do-while Loop

- The condition is checked AFTER the loop body executes (post-test loop)
- The loop body always executes at least once, even if the condition is false
- Syntax

```
do {
// loop body
} while (condition);
```

- Note the semicolon after the closing parenthesis - it is required
- Difference from while loop: the body runs first, then the condition is checked
- Best use case: menus, programs that must perform an action at least once

```
int choice;
do {
cout << "1. Start\n2. Exit\nEnter choice: ";
cin >> choice;
} while (choice != 1 && choice != 2);
```

6.5 Loop Control Statements

- break statement
- Immediately exits the nearest enclosing loop or switch
- Execution continues at the first statement after the loop
- Example: stop searching once a target value is found

```
for (int i = 0; i < 100; i++) {
if (array[i] == target) {
cout << "Found at index " << i;
break;
}
}
```

- continue statement
- Skips the rest of the current iteration and jumps to the update step (in for) or back to the condition check (in while)
- The loop does not exit, it just skips the remaining code for the current pass
- Example: print all numbers except multiples of 3

```
for (int i = 1; i <= 20; i++) {
if (i % 3 == 0) continue;
cout << i << " ";
}
```

- goto statement
- Transfers control to a labeled statement anywhere in the function
- Generally avoided because it makes code hard to follow and maintain
- Almost never used in modern C++

6.6 Nested Loops

- A loop placed inside another loop
- The outer loop runs once; for each outer iteration, the inner loop completes all its iterations
- Total iterations = outer count multiplied by inner count
- Example: print a 3 by 3 grid of asterisks

```
for (int row = 1; row <= 3; row++) {
for (int col = 1; col <= 3; col++) {
cout << "* ";
}
cout << endl;
}
```

- Printing a right-angled triangle pattern

```
for (int i = 1; i <= 5; i++) {
for (int j = 1; j <= i; j++) {
cout << "*";
}
```

```
cout << endl;
}
```

- Multiplication table using nested loops

```
for (int i = 1; i <= 10; i++) {
for (int j = 1; j <= 10; j++) {
cout << setw(5) << i * j;
}
cout << endl;
}
```

6.7 Common Loop Patterns

- Sum of first N natural numbers: add i to a running total from 1 to N
- Factorial of N: multiply i into a running product from 1 to N
- Fibonacci series: each term is the sum of the previous two terms
- Check if a number is prime: try dividing by all numbers from 2 to square root of N
- Reverse digits of a number: use modulus to extract the last digit and build the reversed number
- Sum of digits: repeatedly extract last digit with % 10 and divide by 10
- Count occurrences of a digit in a number
- Print number triangle, star pyramid, and diamond patterns